Review of Research Paper number 24

Lucien Le Gall, Noé Casteleyn, Loris Manganelli

version: March 31, 2026

1 Introduction

The Python code for Section 2.2 is available in the file ADMM_FHE.py, and its execution is provided in the accompanying notebook. The project repository can be accessed at:

`https://github.com/Loris-Manganelli/projet-cooperative-optimisation/tree/main`

1.1 Summary of the paper

The article [1] focuses on privacy preserving *via* encryption (which is one of three ways among anonymisation, encryption, and differential privacy to ensure data privacy). It mainly says that existing approaches of encryption are cloud-based solution. Thus, the authors have the original idea to combine peer-to-peer (what they call cooperative) ADMM with encryption (based on a fully homomorphic encryption (FHE) scheme).

1.2 Problem Formulation

The article focuses on the following problem:

$$\min_{z_i} \quad \sum_{i=1}^{M} f_i(z_i, p_i) \text{ s.t.} \qquad z_i = (\zeta)_{\kappa_i}$$

where M is the number of agents, and ζ a variable that enables consensus. The function f_i is defined as:

$$f_i(z_i, p_i) = \frac{1}{2} z_i^T H_i z_i + p_i^T F_i^T z_i + \begin{cases} 0, & \text{if } G_i z_i = E_i p_i, \\ \infty, & \text{otherwise.} \end{cases} \tag{1}$$

The affine constraint ensure that the dynamic is feasible, it is a link between the variables and the states (useful in robotic applications).

The articles introduces the following notation: $z_i = \begin{pmatrix} \alpha_i \\ \gamma_i \end{pmatrix}$, and $\rho_i = \begin{pmatrix} \beta_i \\ \delta_i \end{pmatrix}$. The variable α_i is the agent's variable, while γ_i represents other information the agent have over its neighbors.
We can adapt the course main problem

$$\min_{x \in \mathbb{R}^n} \sum_{i=1}^{N} f_i(x) \tag{2}$$

with the following formulation:

$$\begin{aligned} &\min_{\alpha_i \in \mathbb{R}^m, \gamma_{ij} \in \mathbb{R}^m} \sum_{i=1}^{N} f_i(\alpha_i) \\ \text{s.t } &\gamma_{ij} = \zeta_j \quad \forall i \sim j && \textit{(i)} \\ &\alpha_i = \zeta_i \quad \forall i \in M && \textit{(ii)} \\ &\alpha_i = \gamma_{ij} \quad \forall i \sim j && \textit{(iii)} \end{aligned} \tag{3}$$

With $G_i = \begin{pmatrix} Id_m & -Id_m & 0 & \dots & 0 \\ Id_m & 0 & -Id_m & \dots & 0 \\ \vdots & & & & \vdots \\ Id_m & 0 & 0 & \dots & -Id_m \end{pmatrix}$, $z_i = \begin{pmatrix} \alpha_i \\ \gamma_{ij} \forall i \sim j \end{pmatrix}$, $E_i = 0$ $\hat{H}_i = \begin{pmatrix} H_i & 0 \\ 0 & 0 \end{pmatrix}$ and $\hat{F}_i^T = \begin{pmatrix} F_i^T & 0 \end{pmatrix}$, we retrieve and equivalent formulation of (2) in the formalism of the article studied.
The problem statement is to derive an encrypted ADMM to fulfill the following security requirements:

1. Only agent i should have access to α_i, H_i, F_i, ρ_i.
2. Neither the agent nor the operator should have access to plaintext values of γ_i
3. Neither the agent nor the operator should have access to plaintext values of ζ_i, tough the agent may know the entries where it is involved.

1.3 Assumptions

Appart the definition of the function for each agent, there is no particular assumptions. We are in the good setting of convex, strongly convex (if H_i is positive definite) and smooth function, so linear convergence holds for peer-to-peer ADMM.

2 Result: theory and practice

2.1 Theory

Let derive the augmented Lagrangian for each device :

$$L_\rho(z_1, \dots, z_N, \zeta, \lambda_1, \dots, \lambda_N) := \sum_{i=1}^{N} \left(\frac{1}{2} z_i^T H_i z_i + y_i^T F_i z_i + i_{G_i z_i = 0} + \lambda_i^T \big(z_i - (\zeta)\kappa_i\big) + \frac{\rho}{2} \left\| z_i - (\zeta)\kappa_i \right\|_2^2 \right) \tag{4}$$

with the dual variables $\lambda_i \in \mathbb{R}^{md_i}$ and the positive weighting factor $\rho \in \mathbb{R}$. The three ADMM iterations are:

$$z_i^{\tau+1} := \arg\min_{z_i} L_\rho\big(z_1, \dots, z_M, \zeta^\tau, \lambda_1^\tau, \dots, \lambda_M^\tau\big), \tag{5a}$$

$$\zeta^{\tau+1} := \arg\min_{\zeta} L_\rho\big(z_1^{\tau+1}, \dots, z_M^{\tau+1}, \zeta, \lambda_1^\tau, \dots, \lambda_M^\tau\big), \tag{5b}$$

$$\lambda_i^{\tau+1} := \lambda_i^\tau + \rho\big(z_i^{\tau+1} - (\zeta^{\tau+1})\kappa_i\big). \tag{5c}$$

The first step (5a) can be distributed, with intermediary quadratic optimization problem under the linear constraint

$$\begin{aligned} \min_{z_i} \quad & \frac{1}{2} z_i^T H_i z_i + y_i^T F_i^T z_i + \lambda_i^{\tau,T} z_i + \frac{\rho}{2} \left\| z_i - (\zeta^\tau)\kappa_i \right\|_2^2 \\ \text{s.t.} \quad & G_i z_i = 0 \end{aligned}$$

and we consequently find $z_i^{\tau+1}$ from solving

$$\begin{pmatrix} H_i + \rho I_{v_i} & G_i^T \\ G_i & 0 \end{pmatrix} \begin{pmatrix} z_i^{\tau+1} \\ \mu_i^{\tau+1} \end{pmatrix} = \begin{pmatrix} \rho(\zeta^\tau)\kappa_i - F_i y_i - \lambda_i^\tau \\ 0 \end{pmatrix}, \tag{6}$$

μ_i values are irrelevant for the algorithm. After few computation, similar to the point 4.5.1 of the lecture notes, we find for the second step (5b) when $\lambda_i^0 = 0$:

$$\zeta_i^{\tau+1} = \frac{1}{d_i} \sum_{j \sim i} \gamma_{ji}^{\tau+1} \tag{7}$$

So one can derive the ADMM iterations:

1. **Computation step** Each device compute $z_i^{\tau+1}$ thanks to 8
2. **Communication step** Each device communicates $\gamma_{ij}^{\tau+1}$ to its neighbors $j \sim i$. And receive those informations from the neighbors.
3. **Computation step** Each device comput ζ_i thanks to 9
4. **Communication step** Each device communicates $\zeta_i^{\tau+1}$ to its neighbors $j \sim i$. And receive those informations from the neighbors.
5. **Computation step** Compute $\lambda_i^{\tau+1}$ as in (5c)

Observations This algorithm only requires an initial guess for the (ζ_i), which can be taken as the average of the initial guesses of the neighboring optimal points. The computational steps clearly correspond to ADMM, allowing us to implement it in a distributed manner.

For the reformulated problem (3) the previous steps implies that $\forall \tau \quad G_i z_i^\tau = 0$ so that z_i is just the repetition of α_i several times. That is why in this specific case (the practice case), we can simplify the first computation step with:

$$\min_{z_i} \quad \frac{1}{2}\alpha_i^T H_i \alpha_i + y_i^T F_i^T \alpha_i + \lambda_i^{\tau,T}(z_i - (\zeta)_{\kappa_i}) + \frac{\rho}{2}\sum_{j\sim i} \left\|\alpha_i - \zeta_j^\tau\right\|_2^2$$

So

$$(H_i + \rho I_{v_i})\alpha_i^{\tau+1} = \rho(\zeta^\tau)\kappa_i - F_i y_i - \lambda_i^\tau \tag{8}$$

the second computation step can be simplified as well:

$$\zeta_i^{\tau+1} = \frac{1}{d_i}\sum_{j\sim i} \alpha_j^{\tau+1} \tag{9}$$

The article described how homomorphic encryption (HE) is used to implement the ditributed ADMM in a privacy-preserving way. The original update steps (e.g., $z_i^{\tau+1}$, $\zeta^{\tau+1}$, $\lambda_i^{\tau+1}$) are computed over encrypted data, ensuring that agents cannot access plaintext values. Homomorphic encryption allows computations directly on ciphertexts, satisfying

$$\mathrm{Dec}(\mathrm{Enc}(x) \oplus \mathrm{Enc}(y)) = x + y, \quad \mathrm{Dec}(\mathrm{Enc}(x) \otimes \mathrm{Enc}(y)) = xy,$$

which enables secure evaluation of ADMM updates. In practice, a leveled fully homomorphic encryption (FHE) scheme is used to support a limited number of operations without costly bootstrapping. Data is encoded into a finite ring $\mathbb{Z}_q$ via scaling and rounding:

$$\lfloor s \cdot x \rceil \bmod q,$$

ensuring compatibility with encrypted computations. The choice of q balances computational efficiency, numerical precision, and overflow risk.

We denote $[\![x]\!] = \mathrm{Enc}_0(x)$. Additively homomorphic encryption allows mixed mutliplication between encrypted data $[\![y]\!]_0$ and x:

$$[\![y]\!]_0 \odot x = [\![y]\!]_0 \oplus [\![y]\!]_0 ... \oplus [\![y]\!]_0 \tag{10}$$

many cryptosystem provide an $\odot$ apllication that does not rely on multiple addition.

We now can adpapt the ADMM setps with encryption

1. **Computation step** Each device compute $[\![z_i^{\tau+1}]\!]_0$ with:

$$\begin{aligned}[\![z_i^{\tau+1}]\!]_0 = &\left(\rho\Gamma_i^{11} \odot [\![(\zeta^\tau)_{\mathcal{K}_i}]\!]_0\right) \ominus \left(\Gamma_i^{11} \odot [\![\lambda_i^\tau]\!]_0\right) \\ &\oplus \left((\Gamma_i^{12}E_i - \Gamma_i^{11}F_i) \odot [\![p_i]\!]_0\right)\end{aligned}$$

   where $\begin{pmatrix}\Gamma_i^{11} & \Gamma_i^{12} \\ * & *\end{pmatrix} := \begin{pmatrix}H_i + \rho I_{v_i} & G_i^T \\ G_i & 0\end{pmatrix}^{-1}$ can be computed once.

2. **Communication step** Each device communicates $[\![\gamma_{ij}^{\tau+1}]\!]_0$ to its neighbors $j \sim i$. And receive those informations from the neighbors.

3. **Computation step** Each device comput $[\![\zeta_i]\!]_0$:

$$[\![\zeta^{\tau+1}]\!]_0 = \frac{1}{|\mathcal{I}_k|} \odot \left(\bigoplus_{i \in \mathcal{I}_k} [\![(z_i^{\tau+1})_{k_i}]\!]_0 \right) \tag{11}$$

4. **Communication step** Each device communicates $[\![\zeta_i^{\tau+1}]\!]_0$ to its neighbors $j \sim i$. And receive those informations from the neighbors.

5. **Computation step** Compute $\lambda_i^{\tau+1}$ with:

$$[\![\lambda_i^{\tau+1}]\!]_0 = [\![\lambda_i^{\tau}]\!]_0 \oplus \left(\rho \odot \left([\![z_i^{\tau+1}]\!]_0 \ominus [\![(\zeta^{\tau+1})_{\mathcal{K}_i}]\!]_0 \right) \right) \tag{12}$$

At the end of the algorithm, in this scheme, no agent has access to the global secret key sk_0. Therefore, ciphertexts encrypted under sk_0 cannot be directly decrypted by any agent. Instead, each agent i owns a personal secret key sk_i, ensuring that only agent i can decrypt information intended for it.

To enable the transition from an encryption under sk_0 to an encryption under sk_i, the protocol relies on a key switching mechanism. An auxiliary key $\mathrm{Enc}_i(sk_0)$ is generated, which corresponds to an encryption of the global secret key sk_0 under the public key of agent i. This information allows the conversion of ciphertexts from the encryption space associated with sk_0 to that associated with sk_i, without revealing any secret key.

The value $\mathrm{Enc}_i(sk_0)$ is made available to a neighboring agent $j \in \mathcal{N}_i$. After l iterations of the algorithm, agent i send $[\![\alpha_i^l]\!]_0$ to agent j which uses this auxiliary key to transform a ciphertext initially encrypted under sk_0 into a ciphertext encrypted under sk_i. As a result, agent j obtains $[\![\alpha_i^l]\!]_i$, which is then transmitted to agent i Since this ciphertext is now encrypted under the key sk_i, only agent i can decrypt it and recover the value α_i^l.

The encrypted ADMM scheme is designed to meet security objectives under an The primary goals are achieved as follows:

- Local parameters α_i, H_i, F_i, and G_i remain confidential because they are only processed in their encrypted forms by neighboring agents.
- The privacy of the state variable $\gamma_i j$ is maintained throughout the process, as it is never decrypted during computation.
- Although agents process certain entries of the global variable ζ, no individual agent possesses the secret key sk_0 required for decryption.

This framework ensures that sensitive local data is protected from unauthorized access during the optimization iterations.

However the article spot a weakness in process:

While the scheme is secure under standard assumptions, vulnerabilities may arise if agents deviate from the protocol by sending arbitrary ciphertexts during the final key switch to access information about sk_0. To address this, the computation of the local update is outsourced to a neighboring agent. This requires the encryption of the weighting matrices to satisfy security goals, leading to a revised update rule. Consequently, the standard update is replaced by Equation (13), which leverages homomorphic operations to ensure privacy:

$$[z_i^{\tau+1}]_0 = ([\rho\Gamma_i^{11}]_0 \otimes [(\zeta^{\tau})_{\mathcal{K}_i}]_0) \oplus ([\Gamma_i^{11}]_0 \otimes [\lambda_i^{\tau}]_0) \oplus ([\Gamma_i^{12}E_i - \Gamma_i^{11}F_i]_0 \otimes [p_i]_0) \tag{13}$$

This modification involves encrypted multiplications and slightly increases communication complexity between neighbors, but it fundamentally prevents agents from presenting unintended data during the key switch.

2.2 Practice

Kernel Ridge Regression For 5 agents and n data points, let us define:

$$f_i(\alpha) = \frac{\sigma^2}{5}\frac{1}{2}\alpha^T K_{mm}\alpha + \frac{1}{2}\|y_i - K_{(i)m}\alpha\|^2 + \frac{\nu}{10}\|\alpha\|^2. \tag{14}$$

Here, $m = \sqrt{n}$ denotes the dimension of α_i.

Our problem differs from the one presented in the article, since only agent i contributes to f_i. In particular, only α_i appears in the function f_i. Therefore, we can define $z_i = \alpha_i$, with no additional information from neighboring agents encoded in z_i. Consequently, we introduce a global variable $\zeta \in \mathbb{R}^m$ to enforce consensus.

Notation. In the original problem formulation, y was a vector containing all n target values, and y_i denoted the i-th coordinate of y (i.e., a scalar). We modify this notation: here, y_i denotes the subset of data associated with agent i (of size n/N, hence a vector). Similarly, $K_{(i)m}$ has size $(n/N) \times m$.

With this convention, the term

$$\frac{1}{2}\sum_{i\in A}(y_i - K_{(i)m}\alpha)^2$$

(which was incorrectly written in the subject as $\frac{1}{2}\sum_{i\in A}\|y_i - K_{(i)m}\|^2$) can be properly rewritten as

$$\frac{1}{2}\|y_i - K_{(i)m}\alpha\|^2.$$

The functions f_i can then be expressed as:

$$f_i(\alpha_i) = \frac{1}{2}\alpha_i^T\left[\frac{\sigma^2}{5}K_{mm} + \frac{\nu}{5}I_m + K_{(i)m}^T K_{(i)m}\right]\alpha_i - y_i^T K_{(i)m}\alpha_i - \frac{1}{2}y_i^T y_i. \tag{15}$$

The constant term $y_i^T y_i$ can be omitted in the optimization process.

Using the notation of the article, we define

$$z_i = \left(\alpha_i\right),$$

and impose the global consensus constraint $z_i = \zeta$.

We obtain:

$$f_i(z_i) = \frac{1}{2}z_i^T H_i z_i - y_i^T K_{(i)m}\alpha_i,$$

with

$$H_i = \frac{\sigma^2}{5}K_{mm} + \frac{\nu}{5}I_m + K_{(i)m}^T K_{(i)m}.$$

Important observation. At this stage, our formulation is very close to a cloud-based version of ADMM (tough the article was focusing on peer-to-peer), where ζ effectively plays the role of a central server. This is clearly a drawback of our implementation, and a compromise must be made.

The classical ADMM iterations are then:

$$z_i^{\tau+1} = (H_i + \rho I)^{-1}\left(K_{(i)m}^T y_i + \rho\zeta^\tau - \lambda_i^\tau\right), \tag{16}$$

$$\zeta^{\tau+1} = \frac{1}{N}\sum_{i=1}^{N} z_i^{\tau+1}, \tag{17}$$

$$\lambda_i^{\tau+1} = \lambda_i^\tau + \rho\left(z_i^{\tau+1} - \zeta^{\tau+1}\right). \tag{18}$$

The article relies on the OpenFHE library to implement the CKKS scheme (a fully homomorphic encryption scheme). This library is primarily designed for C++ and is somewhat outdated for Python. Therefore, we instead use the TenSEAL library as an alternative.

The mapping into a finite ring is treated as a black box: it is convenient to use, although not fully transparent from a theoretical standpoint.

The sketch of our algorithm is the following:

1. Initialization of λ_0 and ζ_0 to 0. The article states that ζ should be initialized with a good estimate of α; here, we assume that 0 is a reasonable initial estimate.
2. Encryption of λ_0 and ζ_0.
3. Computation of z_0 for all agents in the encrypted domain.

4. Iteration in the encrypted domain. Note that the update of λ is performed in the decrypted domain, using the decrypted value of ζ. This is permissible since each agent i has access to the components it is involved in (this also enable to reduce noise and scaling issues).

We did not manage to implement the key switching, but it was useless here.

The result is illustrated below for the parametr $\rho = 1$ and 1000 iterations (the encrypted algorithm is slower in term of computation time than the classical one).

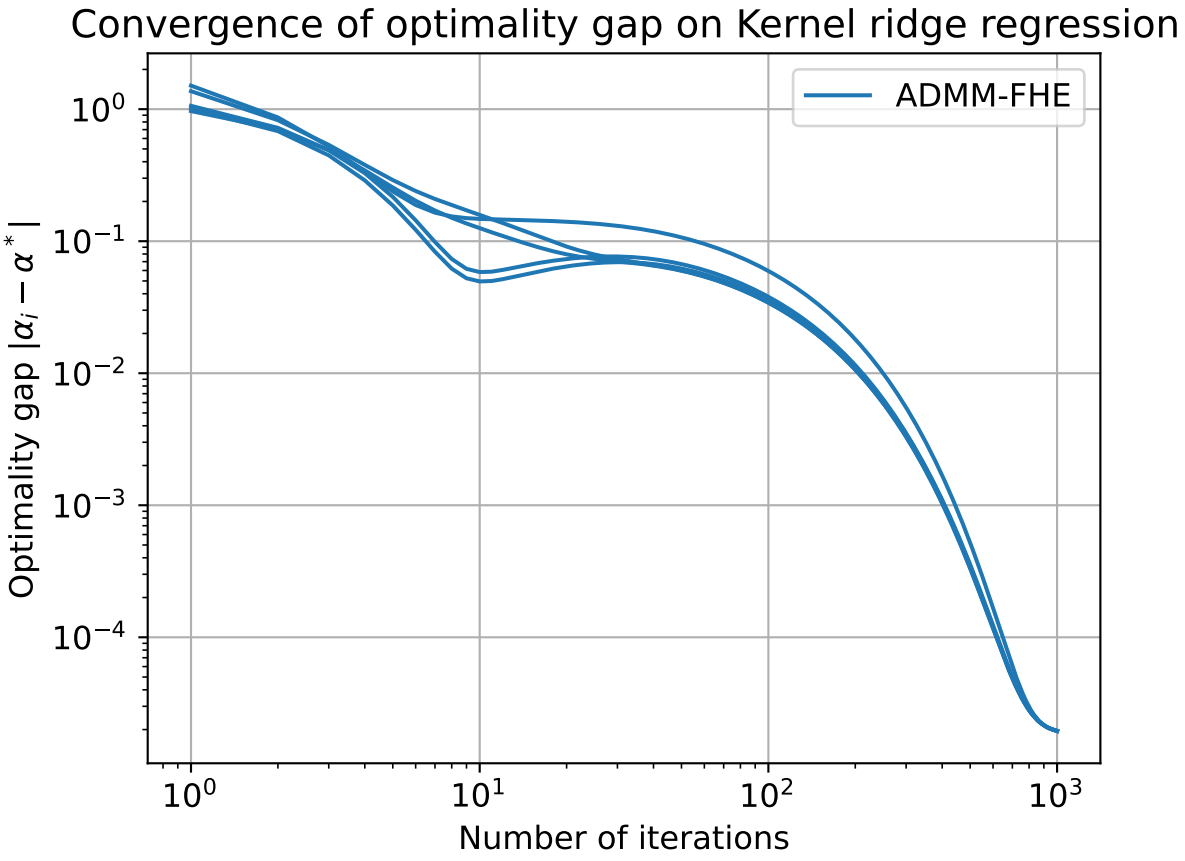

Figure 1: ADMM with FHE on kernel regression problem

3 Conclusion

The added value of ADMM-FHE for our problem is quite limited, since each agent has its own objective function that does not depend on the others. As a result, the ADMM scheme obtained using the formulation from the article is very close to a cloud-based implementation, where ζ effectively plays the role of a central server. This contrasts with the original goal of the article, which was to design an encrypted peer-to-peer ADMM scheme.

Nevertheless, we implemented encrypted ADMM iterations, and the results are quite promising.

References

[1] Philipp Binfet, Janis Adamek, Nils Schlüter, and Moritz Schulze Darup. Towards privacy-preserving cooperative control via encrypted distributed optimization. *at - Automatisierungstechnik*, 71(9):736–747, September 2023.